

Міністерство освіти і науки України
Національний університет "Львівська політехніка"

Кафедра СКС



Лабораторна робота №3
З дисципліни «**АЛГОРИТМИ ТА МОДЕЛІ ОБЧИСЛЕНЬ**»
На тему : **АЛГОРИТМИ ПОШУКУ У МАСИВІ**
Варіант №10

Підготував ст. гр. КІ-203:
Мишак М.А.
Прийняв:
Кіцера А.О.

Львів 2026

Мета роботи: вивчення основних алгоритмів пошуку у масиві.

Завдання:

3	1. Лінійний пошук. Згенерувати одновимірний масив 80 випадкових цілих чисел. Ввести з консолі шуканий елемент. Знайти перше входження шуканого елемента у масиві. Потім в окремому циклі знайти останнє входження елемента. 2. Лінійний пошук. Створити у коді одновимірний масив з 24 додатних і від'ємних цілих чисел серед яких є однакові. Знайти кількість елементів у масиві, які співпадають з першим елементом. 3. Бінарний пошук. Згенерувати одновимірний масив 100 випадкових цілих чисел. Відсортувати масив методом швидкого сортування. Вивести відсортований масив на консоль. Ввести з консолі шуканий елемент. Методом бінарного пошуку знайти шуканий елемент.
---	--

Хід роботи:

```
1. import random
```

```
# Генерація масиву з 80 випадкових цілих чисел
```

```
array = [random.randint(-100, 100) for _ in range(80)]
```

```
print("Масив:")
```

```
print(array)
```

```
x = int(input("\nВведіть шуканий елемент: "))
```

```
# Перше входження
```

```
first_index = -1
```

```
for i in range(len(array)):
```

```
    if array[i] == x:
```

```
        first_index = i
```

```
        break
```

```
# Останнє входження (в окремому циклі)
```

```
last_index = -1
```

```
for i in range(len(array)):
```

```
    if array[i] == x:
```

```
        last_index = i
```

```
# Вивід результатів
```

```
if first_index != -1:
```

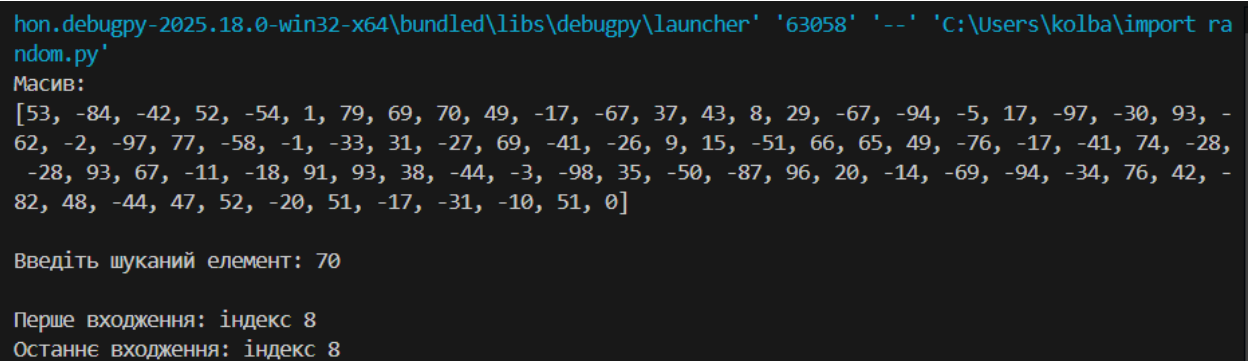
```
    print(f"\nПерше входження: індекс {first_index}")
```

```
    print(f"Останнє входження: індекс {last_index}")
```

```
else:
```

```
    print("\nЕлемент не знайдено в масиві")
```

Скріншот реалізації:



```
hon.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '63058' '--' 'C:\Users\kolba\import random.py'
Масив:
[53, -84, -42, 52, -54, 1, 79, 69, 70, 49, -17, -67, 37, 43, 8, 29, -67, -94, -5, 17, -97, -30, 93, -62, -2, -97, 77, -58, -1, -33, 31, -27, 69, -41, -26, 9, 15, -51, 66, 65, 49, -76, -17, -41, 74, -28, -28, 93, 67, -11, -18, 91, 93, 38, -44, -3, -98, 35, -50, -87, 96, 20, -14, -69, -94, -34, 76, 42, -82, 48, -44, 47, 52, -20, 51, -17, -31, -10, 51, 0]

Введіть шуканий елемент: 70

Перше входження: індекс 8
Останнє входження: індекс 8
```

Рис.1 Реалізація коду №1

2. # Створення масиву з 24 додатніх і від'ємних чисел

```
array = [5, -3, 12, -7, 0, 9, -1, 4, -6, 2,
```

```
        -8, 15, -10, 3, -2, 7, -4, 11, -9, 6,
```

```
        8, -5, 14, -12]
```

```
print("Масив:")
```

```
print(array)
```

```
# Перший елемент
```

```
first_element = array[0]
```

```
count = 0
```

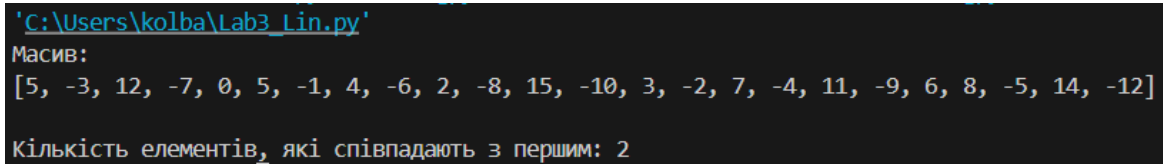
```
for el in array:
```

```
if el == first_element:

    count += 1

print("\nКількість елементів, які співпадають з першим:", count)
```

Скріншот реалізації:



The screenshot shows a terminal window with the following text:
'C:\Users\kolba\Lab3_Lin.py'
Масив:
[5, -3, 12, -7, 0, 5, -1, 4, -6, 2, -8, 15, -10, 3, -2, 7, -4, 11, -9, 6, 8, -5, 14, -12]
Кількість елементів, які співпадають з першим: 2

Рис.2 Реалізація коду №2

3. import random

----- ШВИДКЕ СОРТУВАННЯ -----

```
def quicksort(arr):

    if len(arr) <= 1:

        return arr

    pivot = arr[len(arr) // 2]

    left = []

    middle = []

    right = []

    for x in arr:

        if x < pivot:

            left.append(x)

        elif x > pivot:

            right.append(x)

        else:

            middle.append(x)

    return quicksort(left) + middle + quicksort(right)
```

```
# ----- БІНАРНИЙ ПОШУК -----
```

```
def binary_search(array, x):
```

```
    left = 0
```

```
    right = len(array) - 1
```

```
    while left <= right:
```

```
        mid = (left + right) // 2
```

```
        if array[mid] == x:
```

```
            return mid
```

```
        elif array[mid] < x:
```

```
            left = mid + 1
```

```
        else:
```

```
            right = mid - 1
```

```
    return -1
```

```
# ----- ГОЛОВНА ПРОГРАМА -----
```

```
# Генерація масиву з 100 випадкових чисел
```

```
array = [random.randint(-100, 100) for _ in range(100)]
```

```
print("Початковий масив:")
```

```
print(array)
```

```
# Сортування
```

```
sorted_array = quicksort(array)
```

```
print("\nВідсортований масив:")
```

```
print(sorted_array)
```

```
# Ввід числа

x = int(input("\nВведіть шуканий елемент: "))

# Бінарний пошук

index = binary_search(sorted_array, x)

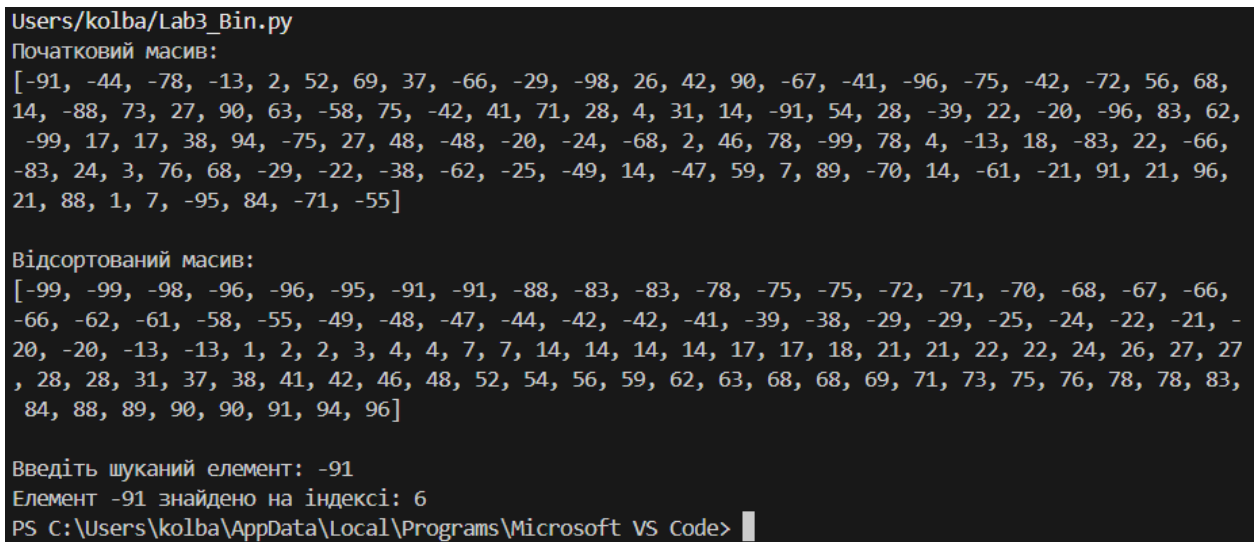
if index != -1:

    print(f"Елемент {x} знайдено на індексі: {index}")

else:

    print("Елемент не знайдено")
```

Скріншот реалізації:



```
Users/kolba/Lab3_Bin.py
Початковий масив:
[-91, -44, -78, -13, 2, 52, 69, 37, -66, -29, -98, 26, 42, 90, -67, -41, -96, -75, -42, -72, 56, 68,
14, -88, 73, 27, 90, 63, -58, 75, -42, 41, 71, 28, 4, 31, 14, -91, 54, 28, -39, 22, -20, -96, 83, 62,
-99, 17, 17, 38, 94, -75, 27, 48, -48, -20, -24, -68, 2, 46, 78, -99, 78, 4, -13, 18, -83, 22, -66,
-83, 24, 3, 76, 68, -29, -22, -38, -62, -25, -49, 14, -47, 59, 7, 89, -70, 14, -61, -21, 91, 21, 96,
21, 88, 1, 7, -95, 84, -71, -55]

Відсортований масив:
[-99, -99, -98, -96, -96, -95, -91, -91, -88, -83, -83, -78, -75, -75, -72, -71, -70, -68, -67, -66,
-66, -62, -61, -58, -55, -49, -48, -47, -44, -42, -42, -41, -39, -38, -29, -29, -25, -24, -22, -21, -
20, -20, -13, -13, 1, 2, 2, 3, 4, 4, 7, 7, 14, 14, 14, 14, 17, 17, 18, 21, 21, 22, 22, 24, 26, 27, 27
, 28, 28, 31, 37, 38, 41, 42, 46, 48, 52, 54, 56, 59, 62, 63, 68, 68, 69, 71, 73, 75, 76, 78, 78, 83,
84, 88, 89, 90, 90, 91, 94, 96]

Введіть шуканий елемент: -91
Елемент -91 знайдено на індексі: 6
PS C:\Users\kolba\AppData\Local\Programs\Microsoft VS Code>
```

Рис.3 Скріншот реалізації коду №3

Висновок: У ході виконання лабораторної роботи №3 я ознайомився з основними алгоритмами пошуку та сортування у масивах, а саме з лінійним пошуком, бінарним пошуком та алгоритмом швидкого сортування. Я реалізував програму, яка генерує масив випадкових цілих чисел, сортує його методом швидкого сортування та здійснює пошук заданого елемента за допомогою бінарного пошуку.

Під час виконання роботи я на практиці переконався, що ефективність алгоритмів суттєво залежить від структури даних: лінійний пошук є простим у реалізації, але неефективним для великих масивів, тоді як бінарний пошук працює значно швидше, проте потребує попереднього сортування масиву. Алгоритм швидкого сортування продемонстрував високу продуктивність і доцільність використання для підготовки даних до пошуку.

Отримані результати дозволили мені краще зрозуміти принципи роботи алгоритмів пошуку та сортування і їх практичне застосування у програмуванні.